

Riwi Shop

Documentacion tecnica del app.js

Explicacion linea x linea y conexion con el HTML

-
1. Conexion inicial — getElementById y addEventListener
 2. Funcion searchProducts — buscar por texto
 3. Funcion getByCategory — filtrar por categoria
 4. Funcion getCatalog — catalogo general
 5. Funcion renderPagination — generar botones de pagina
 6. Funcion crearTarjeta — helper reutilizable
 7. Funcion showAllProducts — boton Ver todo
 8. Mapa completo: HTML a JS

Estas dos lineas son lo primero que ejecuta el navegador al cargar el archivo. Conectan el formulario del HTML con la logica de JavaScript.

Linea de codigo	Que hace	Conecta con el HTML
<pre>const formSearch = document.getElementById('frm-search');</pre>	Busca en el HTML el elemento que tenga id='frm-search'. Lo guarda en la variable formSearch para usarlo en la siguiente linea.	id='frm-search' en el form del HTML
<pre>formSearch.addEventListener('submit', function(event){</pre>	Le dice al formulario: cuando el usuario haga submit (presione Buscar o Enter), ejecuta esta funcion.	Boton type='submit' dentro del form
<pre>event.preventDefault();</pre>	El comportamiento por defecto de un form es recargar la pagina al enviarse. Esta linea lo cancela para que la app maneje el evento.	Sin esto la pagina se recargaria al buscar
<pre>const searchText = this.querySelector('input').value;</pre>	this hace referencia al formulario. querySelector('input') encuentra el campo de texto dentro del form. .value lee lo que escribio el usuario.	input de texto dentro del #frm-search
<pre>if (searchText.trim().length === 0) { return; }</pre>	trim() elimina espacios al inicio y al final. Si lo que queda esta vacio, la funcion termina aqui y no hace ninguna peticion.	Evita busquedas en blanco
<pre>searchProducts(searchText);</pre>	Llama a la funcion searchProducts pasandole el texto que escribio el usuario como argumento.	Dispara la peticion a la API
<pre>this.querySelector('input').value = '';</pre>	Despues de buscar, limpia el campo de texto para que quede vacio y listo para la proxima busqueda.	input de texto del form

Esta funcion hace la peticion a la API con el texto que escribio el usuario y muestra los resultados en el HTML.

Linea de codigo	Que hace	Conecta con el HTML
<pre>async function searchProducts(text) {</pre>	async indica que la funcion va a esperar operaciones que tardan (peticion a internet). Permite usar await adentro.	--
<pre>const searchResults = document.g etElementById('search-results');</pre>	Obtiene la referencia al div donde se van a insertar las tarjetas de resultado.	div id='search-results' en HTML
<pre>const quantity = document.getEle mentById('quantity');</pre>	Obtiene el span donde se muestra el numero de productos encontrados.	span id='quantity' en HTML
<pre>const response = await fetch(`...?q=\${text}`);</pre>	Hace una peticion GET a la API de DummyJSON con el texto buscado en la URL. await pausa la funcion hasta que llega la respuesta.	URL de la API, no toca el HTML
<pre>const data = await response.json();</pre>	La respuesta llega como texto plano. .json() lo convierte en un objeto JavaScript que podemos usar. await espera a que termine.	--
<pre>quantity.textContent = data.total;</pre>	Escribe en el span el numero total de productos que encontro la API. data.total viene del objeto JSON.	span id='quantity' en HTML
<pre>searchResults.innerHTML = '';</pre>	Borra el contenido anterior del div de resultados antes de pintar los nuevos. Sin esto las tarjetas se acumularian.	div id='search-results'
<pre>for (let product of data.products) {</pre>	data.products es un array con los productos encontrados. Este for recorre cada uno de ellos uno por uno.	--
<pre>searchResults.innerHTML += crearTarjeta(product, true);</pre>	Para cada producto llama a crearTarjeta() y agrega el HTML resultante al div. El true activa el rating con estrella.	div id='search-results'

Se ejecuta cuando el usuario elige una categoria del select. Pide a la API solo los productos de esa categoria y maneja la paginacion.

Linea de codigo	Que hace	Conecta con el HTML
<pre>async function getByCategory(categoria, pagina = 1) {</pre>	Recibe el slug de categoria (ej: laptops) y el numero de pagina. pagina=1 es el valor por defecto si no se pasa.	Llamada desde onchange del #select-categoria
<pre>const catalog = document.getElem entById('catalog');</pre>	Obtiene el div del catalogo donde se van a insertar las tarjetas.	div id='catalog' en HTML
<pre>const limit = 30;</pre>	Define cuantos productos se piden por pagina. La API acepta este parametro.	--
<pre>const skip = (pagina - 1) * limit;</pre>	Calcula cuantos productos saltarse. Pagina 1 salta 0, pagina 2 salta 30, pagina 3 salta 60. Asi funciona la paginacion.	--
<pre>await fetch(`...category/\${categ oria}?limit=\${limit}&skip=\${ski p}`);</pre>	Pide a la API los productos de esa categoria. Los parametros limit y skip controlan que pagina se muestra.	URL construida con el valor del select
<pre>catalog.innerHTML = '';</pre>	Limpia el catalogo antes de mostrar los productos de la categoria seleccionada.	div id='catalog'
<pre>catalog.innerHTML += crearTarjeta(product);</pre>	Agrega la tarjeta de cada producto. Sin true porque en el catalogo no se muestra el rating.	div id='catalog'
<pre>renderPagination(data.total, 'getByCategory', categoria);</pre>	Llama a renderPagination pasando el total de productos, el nombre de esta funcion como string, y la categoria.	div id='pagination'
<pre>document.querySelectorAll('#pagi nation button').forEach(btn => { btn.className = '...zinc...'; });</pre>	Selecciona todos los botones de paginacion y los pone en gris (estilo inactivo) para resetear la seleccion anterior.	Botones dentro de #pagination
<pre>document.getElementById(`btn-\${p agina}`).className = 'bg-blue-600 ...';</pre>	Encuentra el boton de la pagina actual por su id y le pone el fondo azul para indicar que esta activa.	btn-{n} dentro de #pagination

Carga todos los productos sin filtro. Se llama automaticamente al abrir la pagina y tambien cuando el usuario hace clic en los botones de paginacion del catalogo general.

Linea de codigo	Que hace	Conecta con el HTML
<pre>async function getCatalog(pagina) {</pre>	Recibe el numero de pagina. A diferencia de getByCategory, no recibe categoria porque muestra todo.	Botones de paginacion del catalogo
<pre>const skip = (pagina - 1) * limit;</pre>	Mismo calculo de paginacion: pagina 1 salta 0 productos, pagina 2 salta 30, etc.	--
<pre>await fetch(`https://dummyjson.c om/products?limit=\${limit}&skip; =\${skip}`);</pre>	Pide todos los productos sin filtrar por categoria. Solo aplica limit y skip para paginar.	--
<pre>catalog.innerHTML = '';</pre>	Limpia los productos anteriores antes de mostrar la nueva pagina.	div id='catalog'
<pre>catalog.innerHTML += crearTarjeta(product);</pre>	Agrega la tarjeta de cada producto al catalogo.	div id='catalog'
<pre>renderPagination(data.total, 'getCatalog');</pre>	Genera la paginacion. Como no hay categoria, solo pasa el total y el nombre de esta funcion.	div id='pagination'
<pre>document.querySelectorAll('#pagi nation button').forEach(...)</pre>	Resetea todos los botones a gris.	Botones en #pagination
<pre>document.getElementById(`btn-\${p agina}`).className = 'bg-blue-600 ...';</pre>	Resalta en azul el boton de la pagina que se esta viendo.	btn-{n} en #pagination

Genera dinamicamente los botones numerados de paginacion. La llaman tanto `getCatalog` como `getByCategory`.

Linea de codigo	Que hace	Conecta con el HTML
<pre>function renderPagination(totalProductos, funcion, categoria = '') {</pre>	Recibe el total de productos, el nombre de la funcion a ejecutar al hacer clic, y opcionalmente la categoria.	Escribe en div id='pagination'
<pre> pagination.innerHTML = '';</pre>	Borra los botones de paginacion anteriores antes de generar los nuevos.	div id='pagination'
<pre> const totalPaginas = Math.ceil(totalProductos / productosPorPagina);</pre>	Divide el total entre 30 y redondea hacia arriba con <code>Math.ceil</code> . Ejemplo: 100 productos / 30 = 3.33 -> 4 paginas.	--
<pre> for (let i = 1; i <= totalPaginas; i++) {</pre>	Crea un boton por cada pagina, empezando desde 1.	--
<pre> onclick="\${funcion}(\${i}\${categoria ? `, '\${categoria}'` : ''})"</pre>	El onclick del boton llama a la funcion correcta con el numero de pagina. Si hay categoria la incluye como segundo argumento.	Boton generado dinamicamente
<pre> id="btn-\${i}"</pre>	Cada boton recibe un id unico: btn-1, btn-2, btn-3... Esto permite encontrarlo despues para cambiarle el color.	Referenciado por getElementById en getCatalog y getByCategory

Esta funcion no existia en el original. El mismo HTML de la tarjeta estaba copiado 3 veces en searchProducts, getByCategory y getCatalog. Se extrajo aqui para escribirlo una sola vez.

Linea de codigo	Que hace	Conecta con el HTML
<pre>function crearTarjeta(product, mostrarRating = false) {</pre>	Recibe un objeto producto y un booleano opcional. mostrarRating controla si se muestra la estrella con el numero.	Usada por searchProducts, getByCategory y getCatalog
<pre>const { title, thumbnail, price, rating } = product;</pre>	Destructuring: en vez de escribir product.title, product.price, etc., extrae esas 4 propiedades en variables directas.	--
<pre>return ` <div class='bg-zinc-900 ... '> ... </div> `;</pre>	Devuelve el HTML completo de la tarjeta como un string. Las clases como bg-zinc-900 son clases de Tailwind CSS que viene del CDN cargado en el HTML.	El string se inyecta en #search-results o #catalog
<pre></pre>	thumbnail y title vienen del objeto producto de la API. Se insertan en el HTML con template literals (\${ }).	Imagen visible en la tarjeta
<pre>\${mostrarRating ? `<span>* \${rating}</span>` : ''}</pre>	Operador ternario: si mostrarRating es true inserta el span con la estrella y el numero. Si es false inserta un string vacio.	Solo visible en resultados de busqueda

Funcion corta que se dispara cuando el usuario hace clic en el boton "Ver todo" del HTML.

Linea de codigo	Que hace	Conecta con el HTML
<code>document.getElementById('select-categoria').value = '';</code>	Resetea el selector de categoria al valor vacio. Sin esto el select quedaria mostrando la ultima categoria elegida aunque ya no este filtrado.	select id='select-categoria'
<code>getCatalog(1);</code>	Carga el catalogo completo desde la primera pagina, borrando cualquier filtro anterior.	div id='catalog' y div id='pagination'

Y al final del archivo, fuera de cualquier funcion:

Linea de codigo	Que hace	Conecta con el HTML
<code>getCatalog(1);</code>	Esta llamada al final del archivo es la que arranca la aplicacion. Cuando el navegador termina de leer el JS, ejecuta esta linea y carga el catalogo automaticamente.	Primera carga visible al abrir la pagina

Cada elemento interactivo del HTML esta conectado a una funcion de app.js. Esta tabla muestra esa relacion de punta a punta.

Elemento HTML	Evento	Funcion JS	Resultado en pantalla
form id="frm-search"	submit (boton Buscar o Enter)	searchProducts(text)	Tarjetas aparecen en div id="search-results"
select id="select-categoria"	onchange (usuario elige categoria)	getByCategory(cat, pag)	Productos filtrados en div id="catalog"
button "Ver todo"	onclick	showAllProducts()	Se resetea el select y recarga div id="catalog"
Botones de paginacion btn-N	onclick (generado por JS)	getCatalog(n) o getByCategory(n,cat)	Pagina n cargada en div id="catalog"
Carga inicial de la pagina	Script se ejecuta al final	getCatalog(1)	Primera pagina del catalogo en div id="catalog"